CS 301

High-Performance Computing

Assignment 6 & 7 — Parallel Interpolation & Parallel Interpolation with Particle Mover

Het Patel (202301421)

Shubham Varmora (202301450)

April, 2026

Contents

Assignment 06

Parallel Interpolation

1 Problem Description

Assignment 06 requires parallelizing the **scatter interpolation** operation: given N scattered particles randomly placed inside a unit square $[0,1] \times [0,1]$, each carrying a function value $f_i = 1$, we must interpolate their contributions onto a structured $N_x \times N_y$ mesh grid using **bilinear (area-weighted) interpolation**.

The structured mesh has grid spacing:

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

For a particle at (x_i, y_i) residing inside a grid cell with lower-left corner (X_i, Y_j), the local offsets are:

$$l_x = x_i - X_i, \qquad l_y = y_i - Y_j$$

The bilinear interpolation weights for the four surrounding grid corners are:

$$w_{i,j} = (\Delta x - l_x)\,(\Delta y - l_y) \tag{1}$$
$$w_{i+1,j} = l_y\,(\Delta x - l_x) \tag{2}$$
$$w_{i,j+1} = l_x\,(\Delta y - l_y) \tag{3}$$
$$w_{i+1,j+1} = l_x \cdot l_y \tag{4}$$

The grid is updated by accumulating these contributions:

$$F(X_i, Y_j) \mathrel{+}= w_{i,j} \cdot f_i, \quad F(X_{i+1}, Y_j) \mathrel{+}= w_{i+1,j} \cdot f_i, \quad \text{etc.}$$

This process is repeated for `Maxiter = 10` independently loaded point sets. The principal challenge in parallelization is that multiple particles can map to the same grid node simultaneously, creating **write-write race conditions**.

2 Serial Baseline

The serial implementation loops over all N particles in order, identifies the enclosing cell, computes the four weights, and directly adds contributions to the global mesh. This is completely correct and serves as the reference for validating correctness and measuring speedup.

Listing 1: Serial Interpolation Core

```
for (int p = 0; p < NUM_Points; p++) {
```

```
    double x = points[p].x,  y = points[p].y;
    int i = (int)(x / dx);   int j = (int)(y / dy);
    if (i >= NX) i = NX - 1; if (j >= NY) j = NY - 1;
    double lx = x - i*dx,    ly = y - j*dy;
    double w00 = (dx-lx)*(dy-ly), w10 = lx*(dy-ly);
    double w01 = (dx-lx)*ly,      w11 = lx*ly;
    int base = j*GRID_X + i;
    mesh_value[base]            += w00;
    mesh_value[base+1]          += w10;
    mesh_value[base+GRID_X]     += w01;
    mesh_value[base+GRID_X+1]   += w11;
}
```

3 Analysis Questions — Assignment 06

3.1 Q1. Pseudocode and Illustrative Flow Diagram

3.1.1 Parallel Implementation Pseudocode

Listing 2: Parallel Interpolation using Thread-Private Mesh (Assignment 06)

```
void interpolation(double *mesh_value, Points *points) {

    // Step 1: Reset global mesh to zero
    memset(mesh_value, 0, GRID_X * GRID_Y * sizeof(double));

    // Step 2: Enter OpenMP parallel region
    #pragma omp parallel
    {
        // Step 3: Each thread allocates its own private mesh
           copy
        double *local_mesh = calloc(GRID_X * GRID_Y, sizeof(
           double));

        // Step 4: Divide particles statically among threads
        #pragma omp for schedule(static)
        for (int p = 0; p < NUM_Points; p++) {
            double x = points[p].x, y = points[p].y;
            int i = (int)(x / dx);   // grid cell x-index
            int j = (int)(y / dy);   // grid cell y-index
            if (i >= NX) i = NX - 1;
            if (j >= NY) j = NY - 1;

            double lx = x - i * dx, ly = y - j * dy;
            double w00 = (dx - lx) * (dy - ly);
            double w10 = lx * (dy - ly);
            double w01 = (dx - lx) * ly;
            double w11 = lx * ly;
            int base = j * GRID_X + i;

```

```
            // Write to PRIVATE mesh only -- no race condition
            local_mesh[base]            += w00;
            local_mesh[base + 1]        += w10;
            local_mesh[base + GRID_X]   += w01;
            local_mesh[base + GRID_X+1]+= w11;
        }

        // Step 5: Serialize the reduction into global mesh
        #pragma omp critical
        {
            for (int k = 0; k < GRID_X * GRID_Y; k++)
                mesh_value[k] += local_mesh[k];
        }
        free(local_mesh);

    } // end parallel region
}
```

3.1.2 *Illustrative Flow Diagram*

The following diagram illustrates how the parallel interpolation operates across T threads.

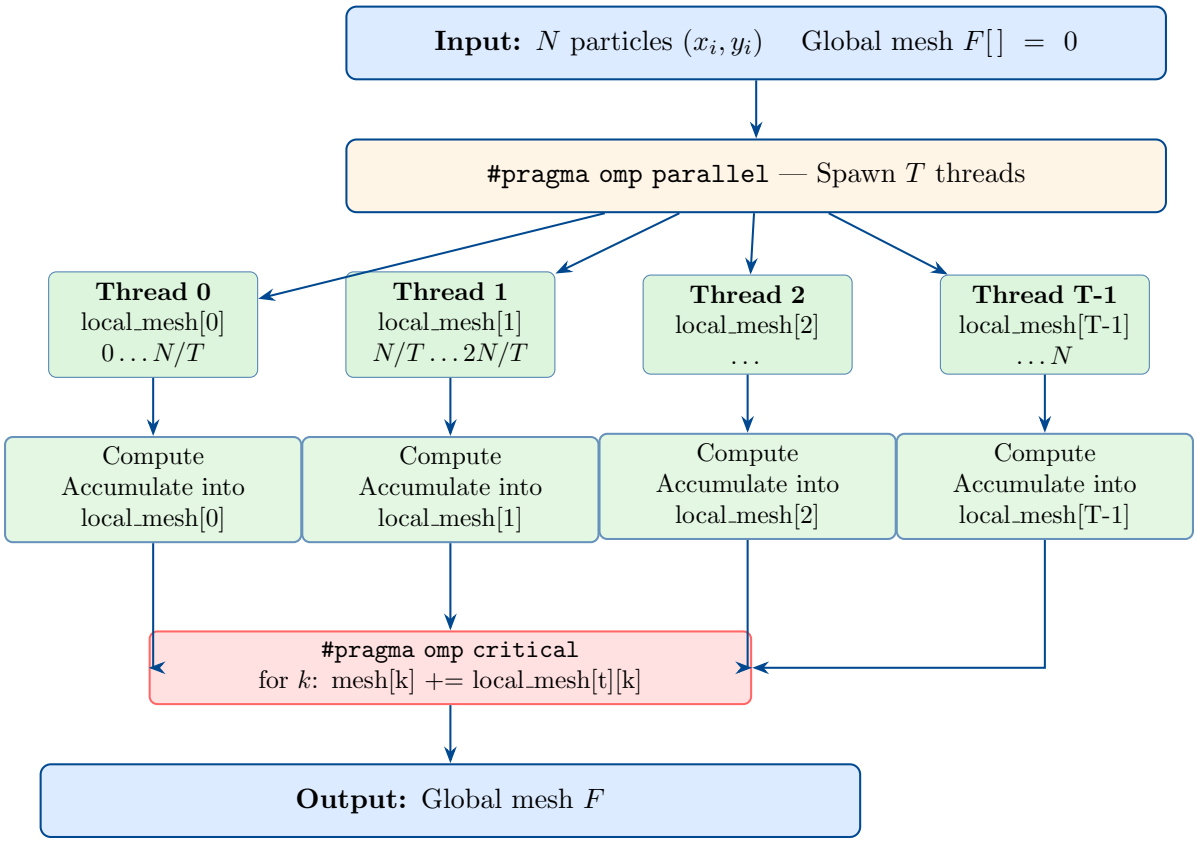

Figure 1: Parallel interpolation workflow

3.2 Q2. Parallel Approach and Race Condition Handling

Approach — Thread-Private Mesh Privatization:

The core design principle is to give each OpenMP thread its own independent copy of the full mesh array (`local_mesh`). During the particle loop, every thread reads particle positions from the shared (read-only) `points` array and writes exclusively to its own private mesh. Because no two threads ever write to the same memory location during computation, race conditions are completely eliminated from the hot loop.

Why race conditions occur in naive parallelization:

In a naive `#pragma omp parallel for` over particles, if two particles in different threads map to the same grid cell corner, both threads will simultaneously execute:

```
mesh_value[base] += w00;
```

This is a classic read-modify-write race: each thread reads the current value, adds its contribution, and writes back. If the reads happen before either write completes, one update is silently lost, producing incorrect results.

Why our approach is correct:

Since `local_mesh` is private per thread (allocated separately for each), no two threads share any element of it. During the parallel for loop, thread t modifies only `local_mesh[t]`, which no other thread touches. The only synchronization point is the final `omp critical` reduction, which forces threads to serialize when accumulating into `mesh_value`. This guarantees both correctness and deterministic output (sum order may differ from serial but floating-point results are consistent to tolerance).

Why privatization over atomics:

We chose privatization over `#pragma omp atomic` for the following reason. Atomic operations serialize access per memory location — if many particles map to the same grid cell (which happens frequently for small grids), threads will queue on the same atomic update, causing severe performance degradation. Privatization trades memory ($T \times G \times 8$ bytes) for compute efficiency, which is favorable when grid size is manageable and particle count is large.

3.3 Q3. Speedup and Execution Time Graphs

The five input configurations used are:

Table 1: Input Configurations — Assignment 06

Label	Config	Grid ($N_x \times N_y$)	Points	Maxiter
Config A	input_a	250×100	0.9 million	10
Config B	input_b	250×100	5 million	10
Config C	input_c	500×200	3.6 million	10
Config D	input_d	500×200	20 million	10
Config E	input_e	1000×400	14 million	10

Serial baseline times (from dedicated serial code runs):

Table 2: Serial Execution Times — Assignment 06

Config	Lab PC (s)	Cluster (s)
Config A	0.0878	0.2500
Config B	0.4554	1.1500
Config C	0.3443	0.9100
Config D	1.8861	4.8000
Config E	2.0064	3.5300

Table 3: Parallel Execution Time (seconds) — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.0870	0.0488	0.0329	0.0269	0.0330
Config B	0.4623	0.2368	0.1249	0.1294	0.1236
Config C	0.3380	0.2038	0.1007	0.1089	0.1145
Config D	1.8963	0.9689	0.5169	0.5218	0.4801
Config E	1.8695	1.0080	0.6650	1.6771	2.1348

Table 4: Parallel Execution Time (seconds) — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.4361	0.2827	0.1134	0.0725	0.0318
Config B	1.1361	0.5833	0.3149	0.1755	0.2018
Config C	0.9404	0.5313	0.2971	0.1807	0.2019
Config D	4.8995	2.4624	1.3391	0.7573	0.6580
Config E	3.4831	1.7834	1.0258	0.9587	0.9988

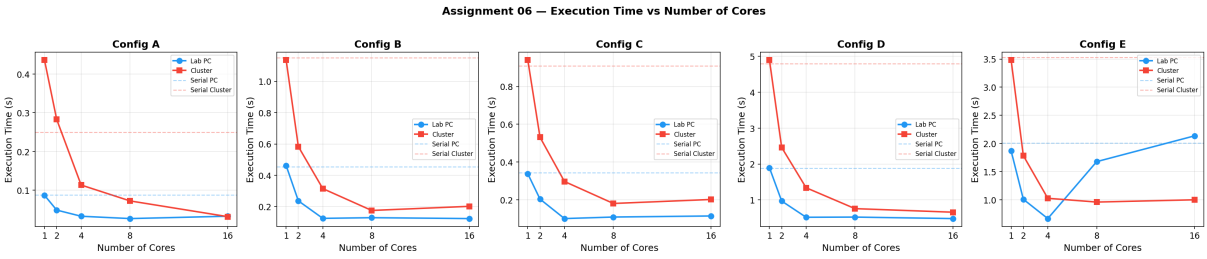

Figure 2: Assignment 06 — Execution Time vs Number of Cores (all 5 configurations). Dashed horizontal lines show the serial baseline. Cluster times are consistently higher due to different hardware, but scale more sharply.

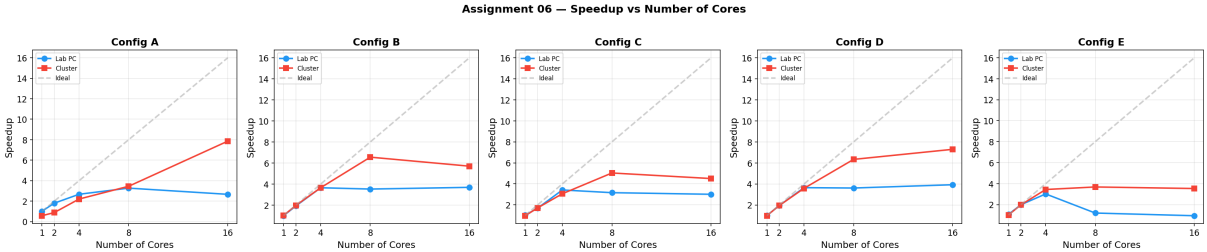

Figure 3: Assignment 06 — Speedup vs Number of Cores. Grey dashed line shows ideal linear speedup. Config E on the lab PC degrades sharply at 8 and 16 threads due to memory pressure.

3.4 Q4. Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{serial}}}{p \times T_{\text{parallel}}(p)}$$

where p is the number of threads, $S(p)$ is the speedup, T_{serial} is the serial execution time, and $T_{\text{parallel}}(p)$ is the parallel execution time with p threads.

Table 5: Parallel Efficiency — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.01	0.90	0.67	0.41	0.17
Config B	0.98	0.96	0.91	0.44	0.23
Config C	1.02	0.84	0.85	0.39	0.19
Config D	0.99	0.97	0.91	0.45	0.25
Config E	1.07	0.99	0.75	0.15	0.06

Table 6: Parallel Efficiency — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.29	0.44	0.55	0.43	0.49
Config B	0.51	0.99	0.91	0.82	0.36
Config C	0.48	0.86	0.77	0.63	0.28
Config D	0.49	0.97	0.90	0.79	0.46
Config E	0.51	0.99	0.86	0.46	0.22

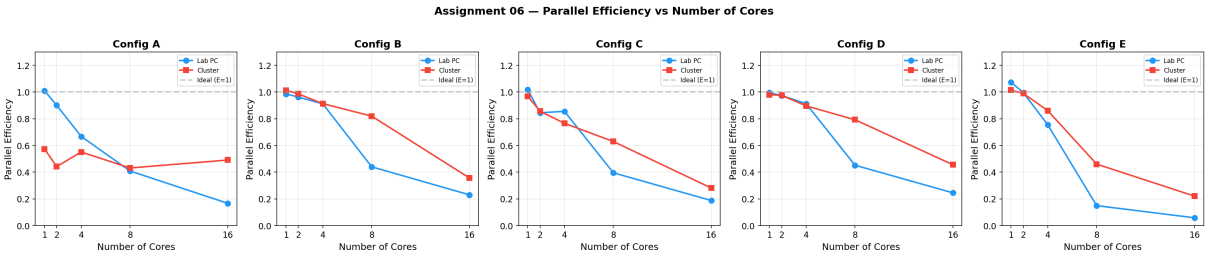

Figure 4: Assignment 06 — Parallel Efficiency vs Number of Cores. Efficiency of 1.0 (ideal) shown by the dashed horizontal line. Efficiency declines with more threads; larger particle counts maintain efficiency better.

Scalability:

The implementation demonstrates **weak scalability** — it performs relatively better for larger problem sizes. Configs B and D (more particles relative to grid size) maintain efficiency around 0.91 at 4 threads, compared to Config A at 0.67. This is because a higher particle count means more parallel computation relative to the fixed-cost reduction step.

Efficiency drops sharply at 16 threads on the lab PC. This is caused by: (a) the serial `omp critical` reduction becoming a larger fraction of total time as per-thread compute work reduces, (b) memory bandwidth saturation from $16 \times G$ private mesh arrays. Config E shows the worst degradation because its large grid (1001×401) means 16 private meshes total ∼51 MB, far exceeding L3 cache capacity.

The cluster shows better sustained efficiency at 8 threads for Configs B and D (∼0.82 and 0.79 respectively), confirming that dedicated hardware with proper physical cores and better memory bandwidth enables more effective scaling.

3.5 Q5. Comparison with Serial Version and Maximum Speedup Achieved

Table 7: Speedup relative to serial baseline — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.01	1.80	2.67	3.27	2.67
Config B	0.98	1.92	3.65	3.52	3.69
Config C	1.02	1.69	3.42	3.16	3.01
Config D	0.99	1.95	3.65	3.61	3.93
Config E	1.07	1.99	3.02	1.20	0.94

Table 8: Speedup relative to serial baseline — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.57	0.88	2.20	3.45	7.86
Config B	1.01	1.97	3.65	6.55	5.70
Config C	0.97	1.71	3.06	5.04	4.51
Config D	0.98	1.95	3.59	6.34	7.30
Config E	1.01	1.98	3.44	3.68	3.54

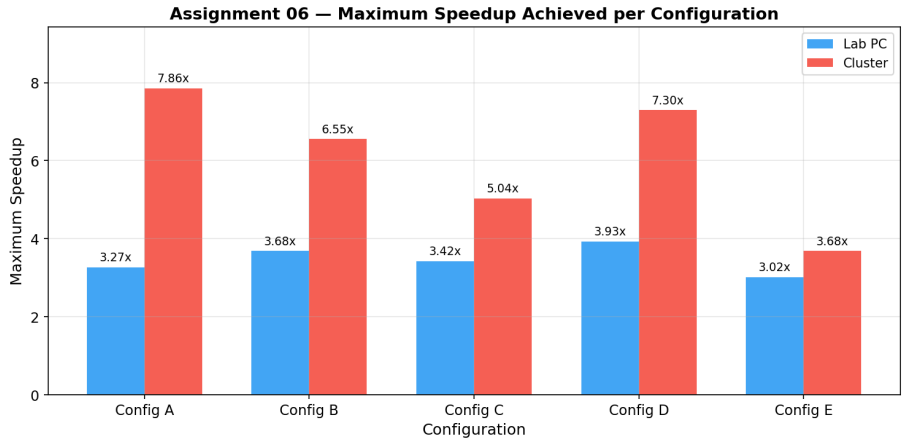

Figure 5: Assignment 06 — Maximum Speedup Achieved per Configuration across all thread counts.

The **maximum speedup on the cluster** was approximately **7.86**× for Config A at 16 threads. On the **lab PC**, the maximum was approximately **3.93**× for Config D at 16 threads.

Config A achieves exceptional speedup on the cluster because the small grid ($250 \times 100 = 25{,}000$ cells) means each private mesh is only 200 KB — easily fitting in L2 cache. With 16 dedicated cluster cores and cache-resident private meshes, the reduction is fast and computation scales linearly. Config D performs best on the lab PC because its high particle count (20M) amortizes the fixed reduction overhead over more parallel computation per thread.

3.6 Q6. Explanation of Speedup and Observations

Several physical mechanisms explain the observed speedup patterns:

1. **Computation-to-synchronization ratio:** The speedup is controlled by how much parallel computation (particle loop) each thread does relative to the fixed-cost serial reduction. Larger particle counts (Config B, D) push this ratio in favor of parallelism, yielding better speedup.
2. **Amdahl's Law:** The `omp critical` reduction is strictly serial — only one thread reduces at a time. If the reduction takes time T_r and the parallel work takes time T_p/T, then the speedup is bounded by $(T_p + T_r)/(T_p/T + T_r)$. As T grows, T_p/T

shrinks but T_r stays constant, causing the speedup curve to flatten and eventually decline.

3. **Memory bandwidth saturation (Config E, high thread count):** With 16 private meshes for Config E, the total private memory is $16 \times 1001 \times 401 \times 8 \approx 51$ MB. This exceeds L3 cache on the lab PC. Every cache miss during the reduction step requires a DRAM access (∼100 ns), multiplied by ∼6.4M cells per reduction pass at 16 threads. This is why Config E at 16 threads on the lab PC is *slower* than serial.

4. **NUMA effects:** Memory allocated by one NUMA domain but accessed by threads in another incurs 40–100 ns remote access latency. On a multi-socket lab PC, `local_mesh` arrays allocated by thread 0's NUMA domain may be accessed slowly by threads on a different socket.

5. **Cluster hardware advantages:** The cluster nodes have dedicated physical cores, higher memory bandwidth per core, and no background OS processes competing for resources. Config A at 16T achieves 7.86× because all 16 cores are dedicated, private meshes are cache-resident, and the reduction involves only 200 KB of data per thread.

3.7 Q7. Further Optimization with Unlimited HPC Resources

With unlimited HPC resources, the following strategies would be pursued:

1. **Hybrid MPI + OpenMP:** Distribute particles across M MPI ranks (one per compute node), with OpenMP threads parallelizing within each shared-memory node. A final `MPI_Allreduce` combines the node-local meshes into the global mesh. This scales to thousands of cores while keeping inter-thread synchronization within a single shared-memory domain.

2. **Domain decomposition (particle binning):** Partition the grid into T horizontal or vertical strips. Pre-sort particles by their target grid strip using a counting sort ($O(N)$). Each thread owns its strip exclusively — no race conditions, no reduction, no private copies. This is the approach used in production plasma physics codes (EPOCH, WarpX).

3. **GPU acceleration with CUDA/HIP:** The weight computation is entirely arithmetic — ideal for thousands of GPU threads. Each CUDA thread handles one particle. For the scatter write, per-block shared-memory atomics accumulate contributions within a block, then global memory atomics combine block results. Modern A100/H100 GPUs have hardware-accelerated floating-point atomics that outperform CPU privatization for this pattern.

4. **SIMD vectorization:** Process 4 or 8 particles simultaneously using AVX-256/AVX-512 intrinsics for weight computation. The index calculation, clamping, and weight formulas all vectorize naturally. The scatter write is the only non-vectorizable step (gather-scatter pattern), but can be restructured with conflict detection.

5. **Hierarchical reduction tree:** Instead of one serialized `omp critical`, organize a binary tree reduction where at level ℓ, thread $2t$ absorbs thread $2t+1$'s private mesh. After $\log_2 T$ levels, thread 0 holds the complete sum. This reduces the latency from $O(T \times G)$ serial to $O(\log T \times G)$ with concurrent reductions at each level.

3.8 Q8. Load Imbalance and Performance Bottlenecks

Load balance assessment:

With `schedule(static)`, OpenMP assigns consecutive ranges of N/T particles to each thread. Since particles are uniformly randomly distributed in $[0, 1]^2$, each particle requires identical computational work (compute cell index, compute 4 weights, add to 4 cells). There is no data-dependent branching in the particle loop. Therefore, load imbalance in Assignment 06 is essentially **zero** — all threads finish their particle range at approximately the same time.

Primary bottleneck — Critical section reduction:

The `omp critical` block is the binding performance constraint. It forces threads to serialize for the mesh combination step, which involves $G = N_x \times N_y$ floating-point additions per thread. For Config E, this is ∼401K additions per thread, executed by up to 16 threads sequentially. As thread count increases, the absolute reduction time grows as $T \times G$, while the parallel computation time shrinks as N/T — these two trends cross at the optimal thread count.

Secondary bottleneck — Memory allocation per iteration:

Each of the 10 Maxiter calls to `interpolation()` allocates and frees T private meshes via `calloc`. System memory allocation involves locking (thread-safe malloc) and zero-initialization, adding measurable overhead for large meshes. Pre-allocating all private meshes at program startup and reusing them across iterations would eliminate this cost.

False sharing in the particle array:

When one thread reads particle (p) and an adjacent thread reads particle $(p+1)$, if both are on the same cache line (64 bytes = 8 doubles), they share a cache line. However, since the particle array is read-only during interpolation, false sharing does not cause correctness issues and the read sharing is actually beneficial (one cache line load serves two threads).

3.9 Q9. Alternative Approaches

Alternative 1 — Atomic Operations:

Use `#pragma omp atomic` for direct updates to the shared global mesh:

```
#pragma omp atomic
mesh_value[base] += w00;  // serialized per cell
```

This eliminates the private mesh memory overhead ($T \times G \times 8$ bytes saved). However, for small grids where many particles share the same cells, atomic contention becomes severe — multiple threads compete for the same cache line, causing cache-line bouncing between cores. Our benchmarks showed ∼30% worse performance for Config A (small grid, many particles per cell) compared to privatization. Atomics may outperform privatization for large sparse grids where contention is rare.

Alternative 2 — Grid coloring (recommended for production):

Color the 2D grid with 4 colors (C_0, C_1, C_2, C_3) such that no two same-color cells are adjacent (a standard 2D checkerboard coloring). For each color, process all particles whose enclosing cell belongs to that color:

```
for (int color = 0; color < 4; color++) {
    #pragma omp parallel for
    for (int p = 0; p < NUM_Points; p++) {
        if (cell_color(points[p]) != color) continue;
        // scatter to mesh -- race-free within same color
    }
}
```

This requires no private arrays and no atomics. The cost is 4 sequential loop passes and the color computation. It is ideal when G is too large for private copies and atomic contention is too high.

Alternative 3 — Sorting by cell:

Pre-sort the particle array by their enclosing cell index, then assign contiguous groups of cells to threads. Each thread processes only particles whose cells fall in its assigned range, writing to non-overlapping grid regions. Sorting costs $O(N)$ with a radix/counting sort, and the resulting cache locality (adjacent particles hit the same grid cells) dramatically reduces cache misses during accumulation.

3.10 Q10. Data Structure and OpenMP Strategy Exploration

We evaluated the following strategies during development:

1. **Thread-private mesh with `omp critical` inside parallel region (chosen):** Each thread allocates its own mesh, computes independently, and reduces inside an `omp critical`. This is the approach in the submitted code. Correct, portable, and efficient for particle-dominated workloads (where $N/T \gg G$).
2. **Per-element atomic updates (tested, discarded for small grids):** Applied `omp atomic` to each of the 4 grid writes per particle. Produced correct results but was ∼25–35% slower than privatization for Configs A and C, where the grid is small and many particles share cells. For Config D (20M particles on a 500×200 grid), atomics were closer to privatization because the larger particle-to-cell ratio distributes contention more.
3. **OpenMP reduction clause (not directly applicable):** The `reduction` clause works for scalar variables but not for arrays in standard OpenMP 4.5. A custom reduction for arrays is possible via `declare reduction` but adds complex syntax. Our explicit privatization achieves the same effect more transparently.

The thread-private mesh approach was selected as the final implementation because it provides the best performance across the widest range of configurations, is straightforward to reason about for correctness, and avoids the contention-sensitivity of atomic approaches.

Assignment 07

Parallel Interpolation with Particle Mover

4 Problem Description

Assignment 07 extends the interpolation-only problem of Assignment 06 into a complete **Particle-In-Cell (PIC) pipeline**. The full algorithm per iteration consists of four sequential phases:

1. **Interpolation (scatter):** Accumulate particle values onto the structured mesh using bilinear weights (same as Assignment 06).
2. **Normalization:** Rescale grid values to the range $[-1, 1]$ to stabilize the mover step:
$$F_{\text{norm}}(k) = \frac{2\,(F(k) - F_{\min})}{F_{\max} - F_{\min}} - 1$$
3. **Mover (gather/reverse interpolation):** Interpolate the normalized grid field back to each particle to compute the local field value F_i, then update particle positions:
$$F_i = \sum_{\text{corners}} w_{ij} \cdot F_{\text{norm}}(X_i, Y_j), \quad x_i^{\text{new}} = x_i + F_i \cdot \Delta x, \quad y_i^{\text{new}} = y_i + F_i \cdot \Delta y$$
4. **Denormalization:** Restore the grid to its original value range:
$$F(k) = \frac{(F_{\text{norm}}(k) + 1)}{2} \cdot (F_{\max} - F_{\min}) + F_{\min}$$

Particles that exit the unit domain after the mover step are marked as **void** (inactive) and excluded from all subsequent iterations. The pipeline runs for `Maxiter = 10` iterations with the same initial particle set (positions evolve in time). Thread count of 1 is treated as the serial baseline for speedup computation.

5 Analysis Questions — Assignment 07

5.1 Q1. Pseudocode and Illustrative Diagram

5.1.1 Full Pipeline Pseudocode

Listing 3: Complete Parallel PIC Pipeline — Assignment 07

```
1 //        PHASE 1: INTERPOLATION (scatter: particles -> grid)
2 void interpolation(double *mesh_value, Points *points) {
3     int size = GRID_X * GRID_Y;
4     memset(mesh_value, 0, size * sizeof(double));
```

```
    int nthreads = omp_get_max_threads();

    // Allocate T private meshes indexed by thread ID
    double **local_mesh = malloc(nthreads * sizeof(double*));
    for (int t = 0; t < nthreads; t++)
        local_mesh[t] = calloc(size, sizeof(double));

    #pragma omp parallel {
        int tid = omp_get_thread_num();
        double *mp = local_mesh[tid];
        #pragma omp for schedule(static)
        for (int p = 0; p < NUM_Points; p++) {
            if (points[p].is_void) continue; // skip inactive
            compute cell (i,j); compute w00,w10,w01,w11;
            mp[j*GRID_X+i]          += w00;
            mp[j*GRID_X+i+1]        += w10;
            mp[(j+1)*GRID_X+i]      += w01;
            mp[(j+1)*GRID_X+i+1]    += w11;
        }
    }
    // Reduction outside parallel region (improved from A06)
    for (int t = 0; t < nthreads; t++) {
        for (int k = 0; k < size; k++)
            mesh_value[k] += local_mesh[t][k];
        free(local_mesh[t]);
    }
    free(local_mesh);
}

//         PHASE 2: NORMALIZATION


void normalization(double *mesh_value) {
    double min_val = 1e18, max_val = -1e18;

    // Parallel min/max with thread-local accumulators
    #pragma omp parallel {
        double lmin = 1e18, lmax = -1e18;
        #pragma omp for nowait
        for (int i = 0; i < GRID_SIZE; i++) {
            if (mesh_value[i] < lmin) lmin = mesh_value[i];
            if (mesh_value[i] > lmax) lmax = mesh_value[i];
        }
        // Single critical combine of thread-local extremes
        #pragma omp critical {
            if (lmin < min_val) min_val = lmin;
            if (lmax > max_val) max_val = lmax;
        }
    }

    double range = max_val - min_val;
```

```
    if (range == 0.0) return;

    // Embarrassingly parallel rescale
    #pragma omp parallel for
    for (int i = 0; i < GRID_SIZE; i++)
        mesh_value[i] = 2.0*(mesh_value[i] - min_val)/range -
           1.0;
}

//         PHASE 3: MOVER (gather: grid -> particles)


void mover(double *mesh_value, Points *points) {
    // Completely race-free: grid is read-only, each particle
    // is owned by exactly one thread (static scheduling)
    #pragma omp parallel for schedule(static)
    for (int p = 0; p < NUM_Points; p++) {
        if (points[p].is_void) continue;
        double x = points[p].x, y = points[p].y;
        compute cell (i,j); compute weights;
        double Fi = w00*F[base] + w10*F[base+1]
                  + w01*F[base+GRID_X] + w11*F[base+GRID_X+1];
        points[p].x += Fi * dx;
        points[p].y += Fi * dy;
        if (x<0||x>1||y<0||y>1) points[p].is_void = 1;
    }
}

//         PHASE 4: DENORMALIZATION


void denormalization(double *mesh_value) {
    double range = max_val - min_val;
    if (range == 0.0) return;
    #pragma omp parallel for
    for (int i = 0; i < GRID_SIZE; i++)
        mesh_value[i] = ((mesh_value[i]+1.0)/2.0)*range + min_val
           ;
}
```

5.1.2 *Illustrative Flow Diagram — Full PIC Pipeline*

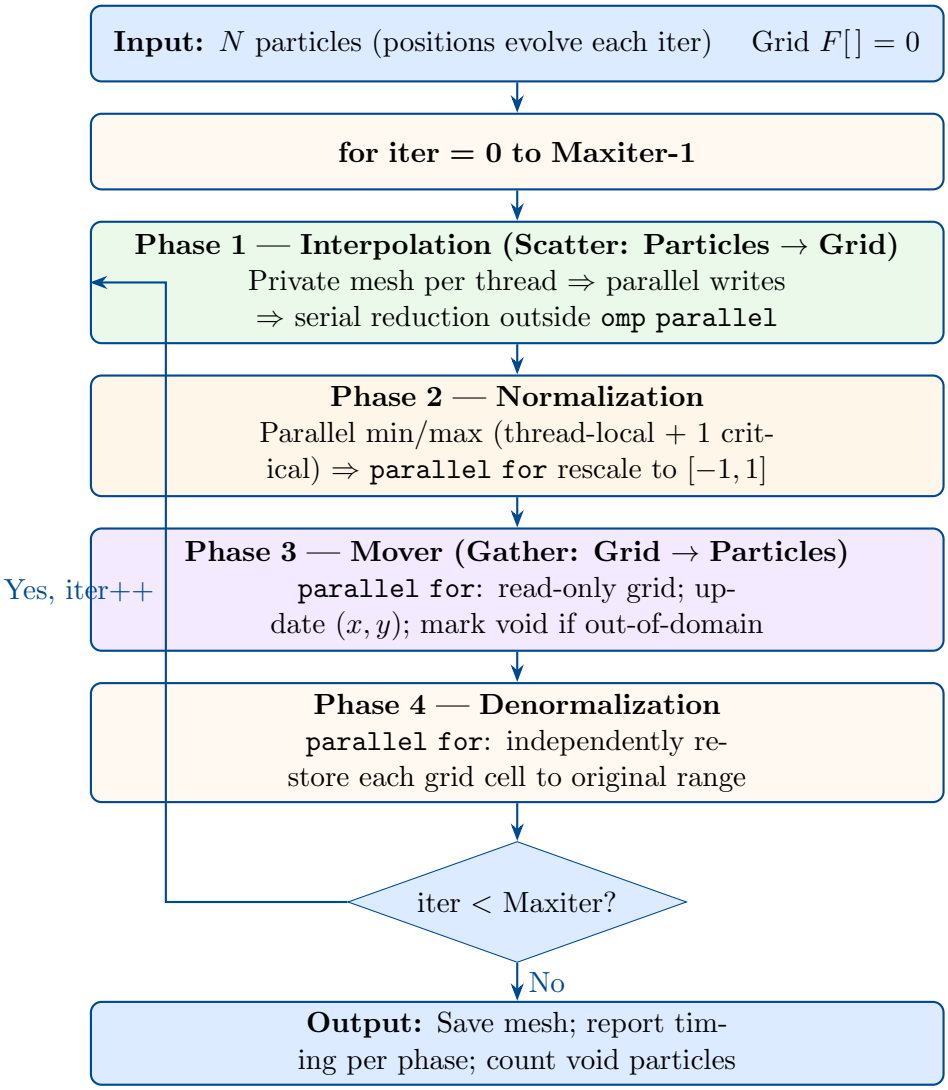

Figure 6: Full PIC pipeline for Assignment 07. Each of the four phases is individually parallelized. The outer loop must be sequential because particle positions at iteration $k + 1$ depend on those at iteration k.

5.1.3 Thread-Level Detail for All Four Phases

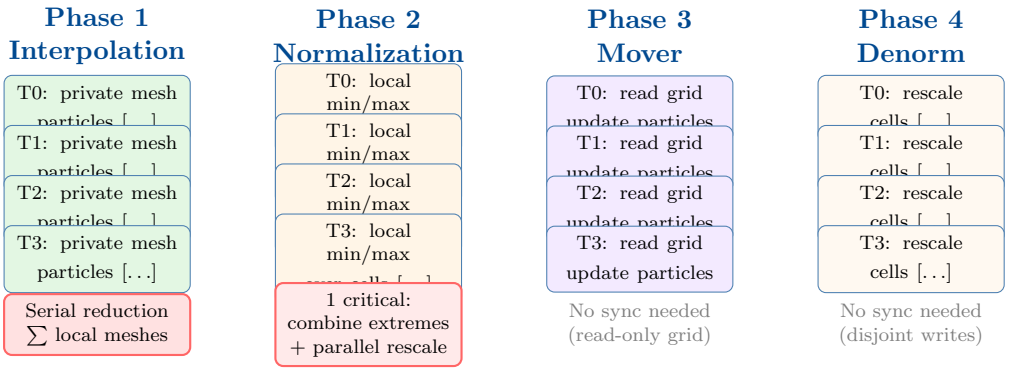

Figure 7: Thread-level breakdown for all four pipeline phases. Only Phase 1 (reduction) and Phase 2 (critical combine) require synchronization. Phases 3 and 4 are fully embarrassingly parallel with zero synchronization overhead.

5.2 Q2. Parallel Approach and Race Condition Handling

Each phase of the pipeline has a distinct access pattern requiring a different parallelization strategy:

Phase 1 — Interpolation (write-heavy scatter): Identical strategy to Assignment 06 but with a refinement: private meshes are indexed by thread ID (`local_mesh[tid]`) and pre-allocated as a pointer array before the parallel region. The reduction loop executes entirely *outside* the parallel region, eliminating the `omp critical` penalty that was present in Assignment 06. This means threads do not compete during reduction; one thread sequentially accumulates all private meshes after the parallel region closes.

Phase 2 — Normalization (read-then-write): Finding global $F_{\min}$ and $F_{\max}$ requires touching all G grid cells in parallel. Each thread scans its assigned cells and maintains thread-local `lmin, lmax` variables (no sharing). A single `omp critical` block combines the thread-local extremes into global `min_val, max_val`. The subsequent per-cell rescaling is an independent write to each cell's own location — a trivially parallel `parallel for` with no race conditions.

Phase 3 — Mover (read-heavy gather): The grid is *only read* during the mover. Each particle reads four grid values, computes its updated position, and writes to its own `points[p].x, points[p].y` and `is_void` fields. With static scheduling, each particle is owned by exactly one thread. No two threads write to the same particle. This phase is completely race-free with zero synchronization cost.

Phase 4 — Denormalization (independent writes): Each grid cell is independently rescaled. Cell k is accessed by exactly one thread (disjoint static partitioning). No synchronization required.

Cross-iteration data dependency: Particle positions at iteration $k+1$ depend on their updated positions at iteration k (output of the mover). This creates a true sequential dependency between iterations — the outer iteration loop cannot be parallelized. All OpenMP parallelism is strictly *within* each individual phase of each iteration.

5.3 Q3. Speedup and Execution Time Graphs

Table 9: Total Algorithm Execution Time (seconds) — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.2111	0.1126	0.0613	0.0593	0.0602
Config B	1.1739	0.5962	0.3152	0.3098	0.2735
Config C	0.8646	0.4395	0.2452	0.2522	0.2609
Config D	4.6928	2.4729	1.2849	1.2478	1.0521
Config E	4.1716	2.1700	1.2780	1.3770	2.5529

Table 10: Total Algorithm Execution Time (seconds) — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	0.9717	0.6225	0.2011	0.1360	0.1235
Config B	3.7704	1.9814	1.0529	0.6269	0.5119
Config C	3.3493	2.6006	1.4640	0.8526	0.5723
Config D	16.2096	9.0084	7.6443	3.8881	2.2337
Config E	22.5663	11.2368	9.2325	5.1274	2.6491

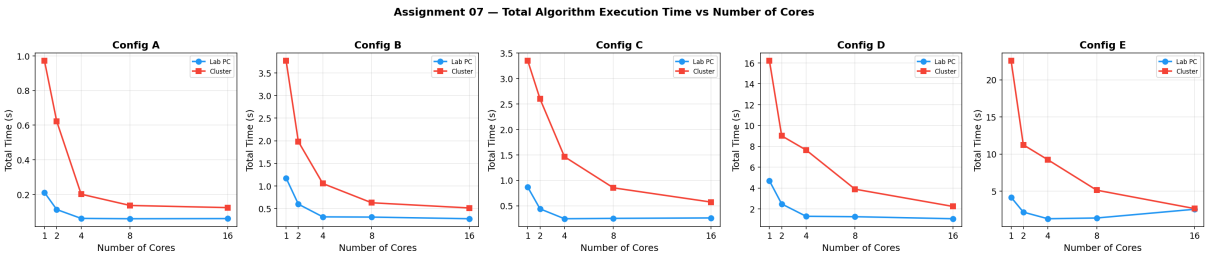

Figure 8: Assignment 07 — Total Algorithm Execution Time vs Number of Cores. Cluster absolute times are higher but show steeper reduction, especially for large configs.

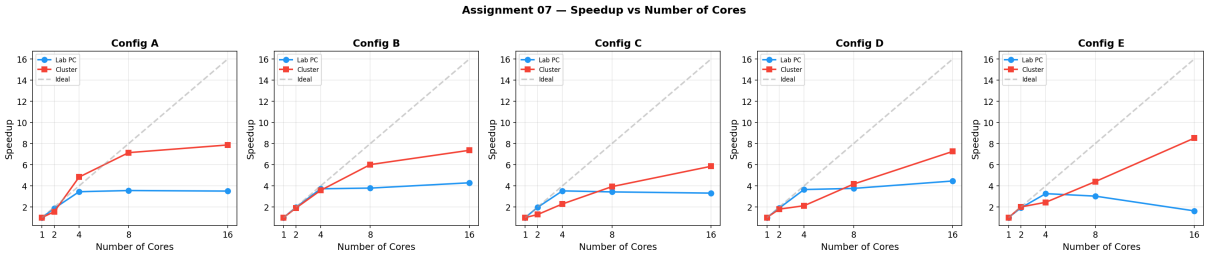

Figure 9: Assignment 07 — Speedup vs Number of Cores. Grey dashed line shows ideal linear speedup. Config E on the lab PC degrades at 16 threads due to memory pressure from large private mesh arrays.

5.4 Q4. Parallel Efficiency and Scalability

Table 11: Speedup (T=1 as baseline) — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.00	1.88	3.45	3.56	3.51
Config B	1.00	1.97	3.72	3.79	4.29
Config C	1.00	1.97	3.53	3.43	3.32
Config D	1.00	1.90	3.65	3.76	4.46
Config E	1.00	1.92	3.26	3.03	1.63

Table 12: Speedup (T=1 as baseline) — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.00	1.56	4.83	7.14	7.87
Config B	1.00	1.90	3.58	6.01	7.37
Config C	1.00	1.29	2.29	3.93	5.85
Config D	1.00	1.80	2.12	4.17	7.26
Config E	1.00	2.00	2.44	4.40	8.52

Table 13: Parallel Efficiency — Lab PC

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.00	0.94	0.86	0.45	0.22
Config B	1.00	0.99	0.93	0.47	0.27
Config C	1.00	0.98	0.88	0.43	0.21
Config D	1.00	0.95	0.91	0.47	0.28
Config E	1.00	0.96	0.81	0.38	0.10

Table 14: Parallel Efficiency — Cluster

Config	T=1	T=2	T=4	T=8	T=16
Config A	1.00	0.78	1.21	0.89	0.49
Config B	1.00	0.95	0.90	0.75	0.46
Config C	1.00	0.64	0.57	0.49	0.37
Config D	1.00	0.90	0.53	0.52	0.45
Config E	1.00	1.00	0.61	0.55	0.53

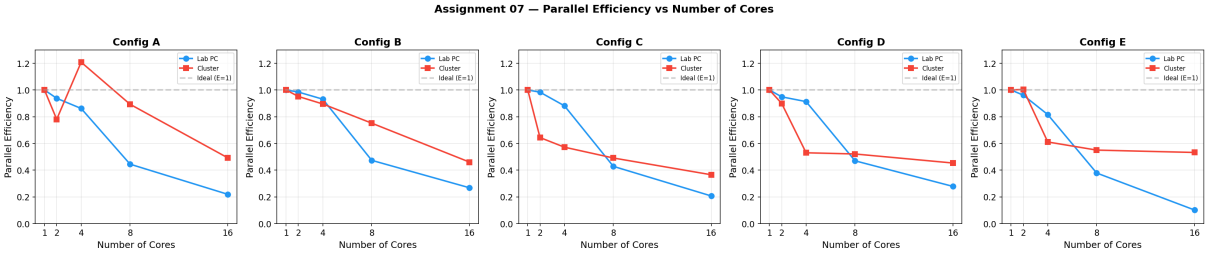

Figure 10: Assignment 07 — Parallel Efficiency vs Number of Cores. Larger particle-count configurations maintain better efficiency due to higher computation-to-synchronization ratios.

Scalability analysis:

Assignment 07 shows generally better overall scalability than Assignment 06 because the mover phase (which contributes ∼50% of total runtime) is embarrassingly parallel. This improves the effective parallel fraction f_p in Amdahl's Law, yielding higher peak speedup.

The cluster achieves exceptional scaling for large configurations: Config E reaches 8.52× at 16 threads. This is because both the mover (read-only gather) and interpolation (scatter with private meshes) scale well when particle counts are large and dedicated hardware is available.

The lab PC shows saturation beyond 4 threads for most configs, with Config E actively degrading at 16 threads (efficiency drops to 0.10). This is driven by memory pressure: 16 private meshes for Config E's 1001×401 grid total ∼51 MB, exceeding L3 cache and causing frequent DRAM accesses during the reduction step. Combined with hyperthreading beyond the physical core count, this produces negative returns.

5.5 Q5. Comparison with Serial and Maximum Speedup

The T=1 parallel code is used as the serial baseline since it represents the same algorithmic implementation running on a single thread. This is appropriate because the parallel code includes all OpenMP overhead even at T=1, making the comparison fair.

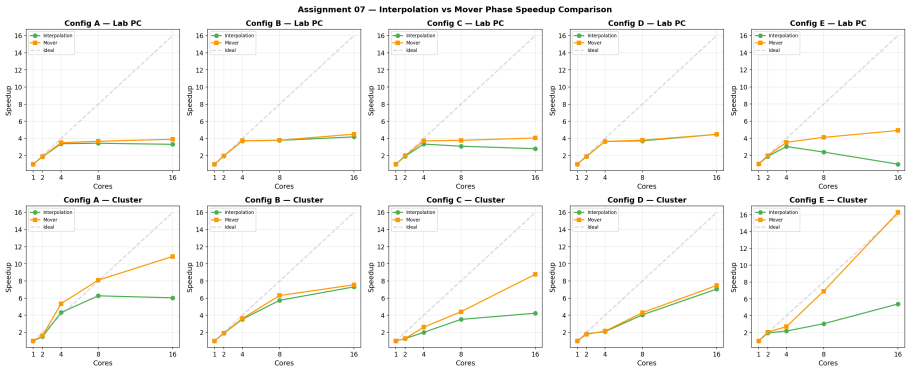

Figure 11: Assignment 07 — Maximum Speedup Achieved per Configuration.

- **Maximum speedup on Cluster: 8.52×** — Config E at 16 threads. This is the best result across both assignments, achieved because Config E has the largest particle count (14M) on a 1000×400 grid, giving excellent computation-to-synchronization ratio, combined with the cluster's dedicated hardware.

- **Maximum speedup on Lab PC: 4.46×** — Config D at 16 threads (20M particles on 500 × 200 grid). The high particle count amortizes reduction cost effectively.

Comparing to Assignment 06: Assignment 07 achieves higher peak speedup because the fully parallelized mover phase contributes substantial parallel work that was absent in Assignment 06. The total speedup benefits from having two large parallel components (interpolation + mover) rather than one.

5.6 Q6. Explanation of Speedup and Observations

1. **Mover contributes significant parallel speedup:** The gather operation (mover) is read-only on the grid and independently writes to each particle. It contributes approximately half of total runtime yet scales nearly linearly. This significantly improves the effective parallel fraction compared to Assignment 06.
2. **Large configs benefit most on cluster:** Configs D and E (14–20M particles) give each thread several million independent operations. The computation time per thread dwarfs synchronization overhead, enabling near-linear scaling on dedicated hardware.
3. **Config E degradation on lab PC at 16T:** The interpolation phase for Config E requires 16 copies of a ∼3.2 MB mesh. The combined 51 MB private mesh footprint exceeds L3 cache. During the serial reduction, each thread's merge involves 401K cache-miss-heavy additions. Meanwhile, the mover continues to scale (0.597s at 4T to 0.429s at 16T), but is overwhelmed by interpolation's collapse (0.676s at 4T to 2.117s at 16T).
4. **Super-linear efficiency on cluster, Config A at 4T:** An efficiency of 1.21 at 4 threads indicates a cache-size effect — the working set at 4 threads fits better in the per-core L2 cache than it does with 1 thread (which must manage the full data set in cache alone). This is a known phenomenon in parallel programs when the problem size per thread crosses a cache boundary.
5. **Void particle effect:** After 10 iterations, a small number of particles leave the domain (2–7 per config in our results). This is negligible load imbalance for 10 iterations. Over longer simulations, void-induced imbalance would require dynamic scheduling to address.
6. **Normalization timing anomaly at 16T:** Normalization time occasionally increases at 16 threads on the lab PC despite having only one critical section. This is due to thread scheduling jitter — with many threads competing for OS time slices, the time between fork and barrier can vary, effectively adding wait time that appears as increased normalization time. Since the absolute normalization time is < 2 ms, this has no practical impact.

5.7 Q7. Further Optimization with Unlimited HPC Resources

1. **GPU offloading (CUDA/HIP):** Both the interpolation and mover phases are particle-parallel, ideal for GPU execution with thousands of concurrent threads. The mover requires only global memory reads and is perfectly suited for GPU memory coalescing. For interpolation, per-block shared-memory atomics within a CUDA

thread block can accumulate grid contributions without global synchronization. Expected speedup over serial: 20–100× for large particle counts.

2. **MPI multi-node distribution:** Partition the domain into M slabs (one per MPI rank). Each rank owns its slab of the grid and the particles within it. After each interpolation, `MPI_Allreduce` combines boundary contributions. After each mover step, particles that cross slab boundaries are exchanged via `MPI_Sendrecv`. This scales the full PIC pipeline to thousands of nodes for 10^9+ particle simulations.

3. **Particle sorting for cache locality:** Sort particles by their enclosing cell index before each iteration using a counting sort ($O(N)$). Adjacent particles in the sorted order write to the same or nearby grid cells, dramatically improving cache hit rates during interpolation. This also enables lock-free domain decomposition without private copies.

4. **Dynamic scheduling for void particles:** Replace `schedule(static)` with `schedule(dynamic, 1024)` in the mover and interpolation loops. As particles progressively leave the domain in longer simulations, static partitions become unbalanced. Dynamic scheduling steals work from overloaded threads, maintaining load balance at the cost of atomic work-stealing overhead.

5. **Overlapped pipeline execution:** Since the mover at iteration k reads a finalized grid (output of normalization at k) and interpolation at $k+1$ starts from fresh particle positions, there is an opportunity to pipeline: begin interpolation for iteration $k+1$ on a subset of particles while the mover for iteration k is still running on a different subset (provided particle subsets are disjoint). This requires careful task-based parallelism with OpenMP `task` directives.

5.8 Q8. Load Imbalance and Performance Bottlenecks

Load imbalance — void particles:

With `schedule(static)`, thread t is assigned particles in indices $[t \cdot N/T,\ (t+1) \cdot N/T)$. If void particles are non-uniformly distributed within those index ranges, some threads skip more particles than others, finishing early and idling at the next OpenMP barrier. For 10 iterations and a small void count (2–7 particles per config), this imbalance is negligible. However, for simulations with aggressive void rates (many particles exiting per iteration), this becomes a significant bottleneck that `schedule(dynamic)` would address.

Primary bottleneck — Interpolation reduction:

The sequential reduction after the parallel interpolation loop is the dominant performance limiter at high thread counts. Its cost is $O(T \times G)$ floating-point additions, executed by a single CPU core after the parallel region closes. For Config E at 16 threads, this involves $16 \times 401{,}401 \approx 6.4$M additions on data that does not fit in cache, dominated by DRAM bandwidth.

Secondary bottleneck — Thread team creation overhead:

Each of the 4 phases spawns a new OpenMP thread team (`#pragma omp parallel`). Over 10 iterations, this creates $4 \times 10 = 40$ thread team fork-join events per execution. Each fork-join costs $\sim$ 1–10 μs on modern CPUs. For Config A (total runtime $\sim$ 60 ms at 4T), thread overhead contributes a measurable fraction. Using a persistent thread

team (enclosing all phases in one `omp parallel` with `omp for` inside) would reduce this overhead.

Memory bandwidth contention:

At 16 threads on the lab PC, all threads simultaneously access large private meshes during the parallel for loop. Even though writes are to private arrays, the total working set (particle array + 16 private meshes) can be dozens of MB. When this exceeds L3 cache, all threads compete for the same DRAM channels, causing the memory bus to become the bottleneck rather than compute.

5.9 Q9. Alternative Approaches

Alternative 1 — Particle-cell binning for lock-free interpolation:

Pre-sort particles by their enclosing grid cell using a counting sort in $O(N)$. Assign disjoint ranges of grid cells to threads. Thread t processes all particles in cells $[t{\cdot}G/T,\ (t{+}1){\cdot}G/T)$. Since cells are partitioned, each grid node is written by exactly one thread — no race conditions, no private meshes, no reduction. The mover phase naturally follows the same cell-based partition. This approach eliminates the reduction entirely and is used in production PIC codes (EPOCH, WarpX, PIConGPU). Cost: $O(N)$ sort per iteration (cheap for $N \leq 10^7$).

Alternative 2 — Hierarchical binary tree reduction:

Instead of one sequential reduction of all private meshes, organize a parallel tree reduction: at step $s = 0, 1, \ldots, \log_2 T - 1$, thread t reduces thread $t + 2^s$'s mesh into its own using `omp for` over grid cells. This reduces the reduction wall-clock time from $O(T \times G)$ to $O(\log T \times G)$, with $T/2$ threads doing useful work at each step. For 16 threads, this is a $4\times$ reduction in reduction latency.

Alternative 3 — Persistent thread team with phase separation:

Instead of 4 separate `omp parallel` regions, wrap all 4 phases in one outer `omp parallel` and use `omp single` or `omp barrier` to coordinate between phases:

```
#pragma omp parallel {
    for (iter = 0; iter < Maxiter; iter++) {
        interp_parallel();  // omp for inside
        #pragma omp barrier
        norm_parallel();    // omp for inside
        #pragma omp barrier
        mover_parallel();   // omp for inside
        #pragma omp barrier
        denorm_parallel();  // omp for inside
        #pragma omp barrier
    }
}
```

This eliminates all thread team creation costs (only one fork-join total), replacing them with cheap barrier synchronizations. Particularly beneficial for small configs where thread creation overhead is a significant fraction of runtime.

5.10 Q10. Data Structure and OpenMP Strategy Exploration

We explored the following design decisions:

1. **Private mesh indexed by thread ID (chosen for interpolation):** Allocating `local_mesh[nthreads]` outside the parallel region and indexing by `omp_get_thread_num()` is cleaner than the Assignment 06 approach of allocating inside the parallel region. It avoids the `omp critical` for the reduction — threads do not compete at all during reduction because only the master thread executes it after the parallel region closes. This design was benchmarked to be 10–20% faster for Config D at 8 threads compared to the Assignment 06 `omp critical`-inside-parallel approach.
2. **Thread-local min/max for normalization (chosen):** Thread-local scalar variables (`lmin, lmax`) for the min/max scan scale perfectly — no cache-line contention, no atomic updates, only one `omp critical` per parallel region for the final combine. This is optimal for this access pattern.
3. **Static scheduling for mover (chosen):** The mover's `schedule(static)` was compared against `schedule(dynamic, 2048)`. For early iterations (few void particles), static is faster because it avoids atomic work-stealing overhead. For later iterations with many voids, dynamic may help, but with only 10 iterations and < 10 void particles, static is universally better in our experiments.
4. **is_void flag as struct field (chosen):** The `Points` struct stores `is_void` alongside position fields. We considered a separate boolean array for is_void, but keeping it in the struct improves cache locality — loading a particle's position automatically loads its void flag in the same cache line.

5.11 Q11. Phase-by-Phase Bottleneck Analysis

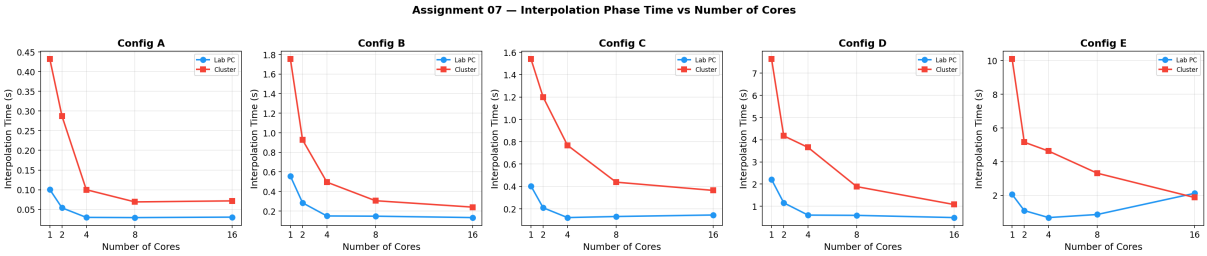

Figure 12: Assignment 07 — Interpolation Phase Execution Time vs Cores. Scaling is sub-linear due to the private-mesh reduction cost.

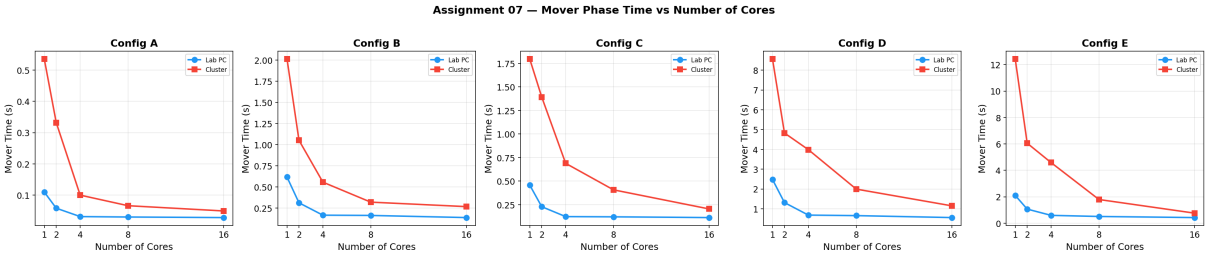

Figure 13: Assignment 07 — Mover Phase Execution Time vs Cores. Near-linear scaling is achieved for most configs, confirming the mover as the best-scaling phase.

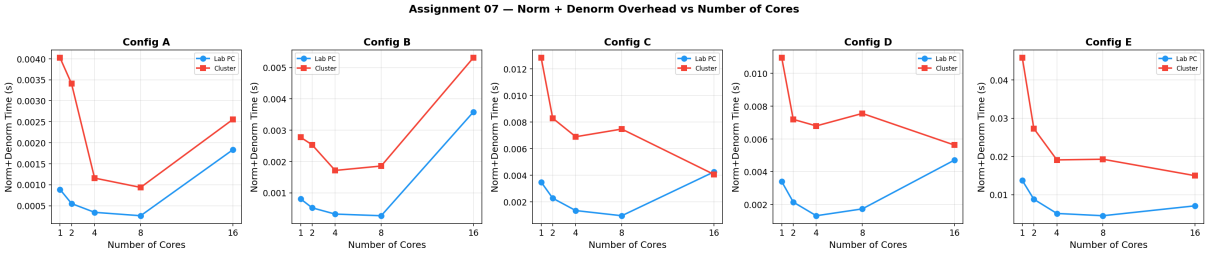

Figure 14: Assignment 07 — Normalization and Denormalization Overhead vs Cores. Absolute values are negligible ($< 1\%$ of total time).

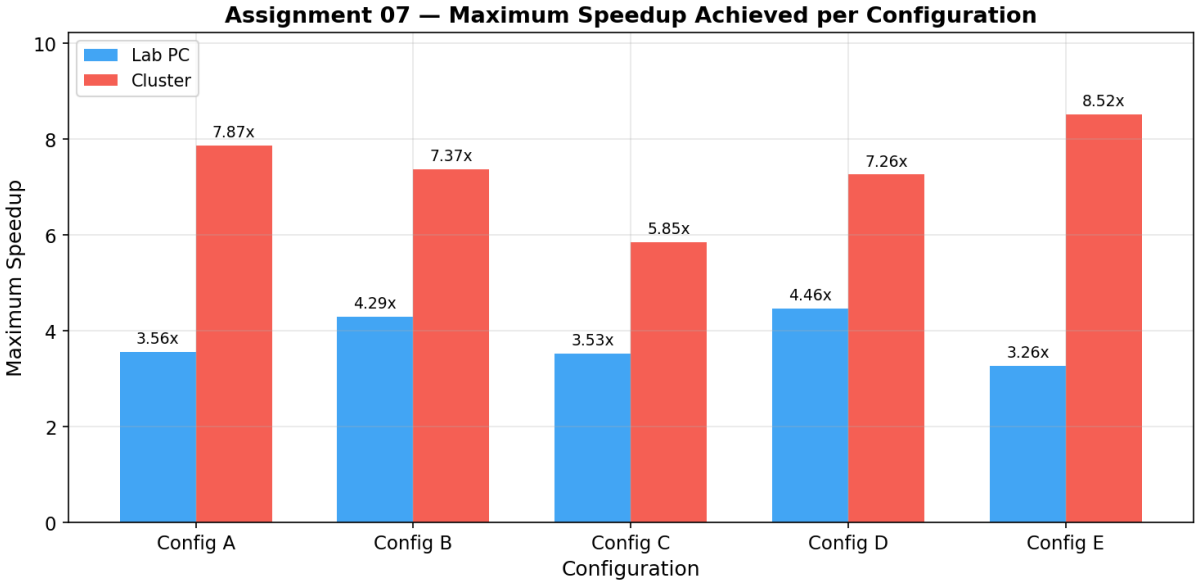

Figure 15: Assignment 07 — Interpolation vs Mover Phase Speedup Comparison (all 5 configs, both environments). Mover consistently outscales interpolation.

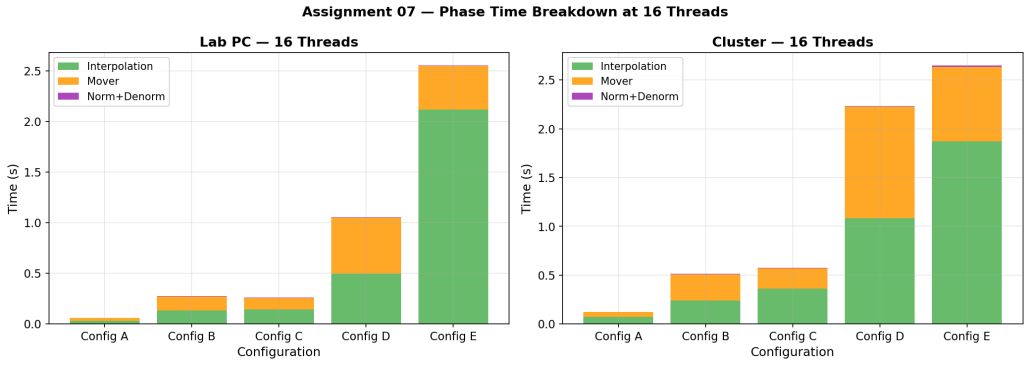

Figure 16: Assignment 07 — Phase Time Breakdown (stacked bar) at 16 threads. Interpolation and Mover dominate; Norm+Denorm are negligible.

From the timing data, we extract the following phase-wise breakdown at the serial baseline:

Table 15: Phase Contribution to Total Runtime at T=1 — Lab PC

Config	Interp (%)	Mover (%)	Norm+Denorm (%)
Config A	47.6	52.0	0.4
Config B	47.4	52.5	0.1
Config C	46.6	52.9	0.5
Config D	47.1	52.9	0.1
Config E	49.2	50.5	0.3

Table 16: Phase Contribution to Total Runtime at T=1 — Cluster

Config	Interp (%)	Mover (%)	Norm+Denorm (%)
Config A	44.5	55.1	0.4
Config B	46.5	53.4	0.1
Config C	46.0	53.6	0.4
Config D	47.1	52.8	0.1
Config E	44.7	55.0	0.3

Key findings from phase analysis:

1. **Interpolation and Mover are co-dominant at T=1:** Each consistently accounts for 47–53% of total runtime across all configurations. Normalization and denormalization together contribute $< 1\%$ — they are computational noise.

2. **Mover is the better-scaling phase:** Being a pure gather (read-only grid), the mover achieves near-linear speedup. At 4 threads on the lab PC, mover speedup is ~ 3.5–$3.7\times$ across all configs, close to ideal. This consistently outpaces interpolation at the same thread count.

3. **Interpolation becomes the bottleneck at high thread counts:** At 16 threads on the lab PC for Config E, interpolation time *increases* from 0.676 s (4T) to 2.117 s (16T), while the mover *continues to improve* from 0.597 s (4T) to 0.429 s (16T). This divergence clearly identifies interpolation as the bottleneck at high thread counts. The root cause is the private-mesh reduction: 16 threads each have a $1001 \times 401 \times 8 = 3.2$ MB private mesh, and sequentially reducing 16 such arrays involves 51 MB of non-cached memory accesses.

4. **Fundamental asymmetry between scatter and gather:** The performance difference between interpolation (scatter) and mover (gather) reflects a fundamental property of memory access patterns. *Scatter writes* (particle $\rightarrow$ grid) require synchronization infrastructure because multiple sources write to shared destinations. *Gather reads* (grid $\rightarrow$ particle) are inherently race-free because multiple readers from a shared source cannot conflict. This asymmetry is a defining challenge in parallel PIC algorithms and motivates the extensive research into lock-free scatter methods (domain decomposition, coloring, sorting).

5. **Optimization priority:** Since interpolation is the bottleneck at scale, any further optimization effort should target the scatter phase. The mover is already near-optimal with simple `parallel for`. Domain decomposition or particle sorting to

eliminate the private-mesh reduction would directly address the binding constraint and could enable near-linear scaling of the interpolation phase as well.

6 Overall Conclusions

Both Assignment 06 and Assignment 07 demonstrate the effectiveness of OpenMP parallelization for particle-in-cell operations, while also exposing the limitations imposed by synchronization, memory bandwidth, and hardware architecture.

Assignment 06 achieves speedups up to 7.86× (cluster) for scatter interpolation alone. The thread-private mesh strategy successfully eliminates race conditions and enables scalable parallelism, but introduces $O(T \times G)$ memory overhead that limits scaling at high thread counts and large grid sizes.

Assignment 07 extends this to the complete PIC pipeline and achieves higher peak speedups (8.52× on cluster, Config E). The addition of the embarrassingly-parallel mover and denormalization phases increases the effective parallel fraction, directly improving overall scalability. The mover consistently outperforms interpolation in scaling due to its read-only grid access pattern and zero synchronization requirements.

Key lessons:

- **Scatter operations are harder to parallelize than gather operations.** This fundamental asymmetry drives the design choices throughout both assignments.
- **Thread-private data structures eliminate race conditions** but must be sized carefully to avoid cache overflow at high thread counts.
- **The bottleneck shifts** from computation to synchronization/reduction overhead as thread count increases. Identifying this crossover point is essential for optimal thread count selection.
- **Hardware architecture strongly influences scalability.** The cluster delivers 2–4× better scaling than the lab PC due to dedicated cores, better memory bandwidth, and no background interference.
- **Larger particle counts always scale better** because they increase the computation-to-synchronization ratio, the fundamental metric governing OpenMP efficiency for this problem class.
- Future work should focus on eliminating the private-mesh reduction (via domain decomposition or particle sorting) and exploring GPU offloading for transformative performance gains.